Analyse discriminante prédictive

Conception et implémentation du package R `ldapkg`

Abdoul-Faisal Ouedraogo

M2 SISE – Programmation R – 2026–2027

Table des matières

1 Objectif et positionnement **2**

2 Modèle statistique **2**

3 Architecture du package **2**

4 Algorithmes et choix d'implémentation **2**

4.1 Calcul efficace 2

4.2 Variables catégorielles 3

4.3 Sélection de variables 3

4.4 Validation 3

5 Sorties interprétables **3**

6 Application Shiny **3**

7 Vérification et reproductibilité **3**

8 Limites et pistes d'amélioration **4**

9 Références **4**

1 Objectif et positionnement

Le projet consiste à fournir une analyse discriminante linéaire prédictive (LDA) utilisable comme un package R installable depuis GitHub. L'objectif est double : produire une décision de classe pour de nouveaux individus et fournir des sorties interprétables sur la séparation des classes, les variables et les axes discriminants.

L'interface publique est une classe R6 inspirée de scikit-learn : `LDAClassifier$new()`, `$fit(X, y)`, `$predict(X)` et `$predict_proba(X)`. Les méthodes `$summary()`, `$plot()`, `$cv()` et `$transform()` complètent l'utilisation analytique et prédictive.

2 Modèle statistique

On note K le nombre de classes, n_k l'effectif de la classe k, m_k son vecteur moyen et m la moyenne globale. Les matrices de sommes des carrés et produits croisés intra-classes et inter-classes sont

$$E = \sum_{k=1}^{K} \sum_{i \in k} (x_i - m_k)(x_i - m_k)^\top, \qquad B = \sum_{k=1}^{K} n_k (m_k - m)(m_k - m)^\top.$$

La matrice totale est $T = E + B$ et la covariance poolée est $W = E/(n - K)$.

Pour une observation x, la fonction de classification de la classe k est

$$\delta_k(x) = x^\top W^{-1} m_k - \frac{1}{2} m_k^\top W^{-1} m_k + \log(\pi_k),$$

où π_k est la probabilité a priori. La classe prédite maximise $\delta_k(x)$. Les probabilités a posteriori sont obtenues par une normalisation softmax stable des scores.

3 Architecture du package

Module	Responsabilité
`preprocess.R`	validation de l'encodage et variables catégorielles
`core.R`	statistiques suffisantes, Cholesky, Wilks, axes
`selection.R`	sélection forward, backward et stepwise
`LDAClassifier.R`	classe R6, apprentissage et prédiction
`methods.R`	méthodes S3 `predict`, `summary`, `plot`
`examples.R`	chargement des jeux de données exemples
`run_app.R`	lancement de l'application Shiny

Le package est limité à des dépendances légères pour le coeur (`R6`, `stats`, `graphics`, `grDevices` et `utils`). Les dépendances de l'application, comme `shiny`, `DT` et `readxl`, sont optionnelles.

4 Algorithmes et choix d'implémentation

4.1 Calcul efficace

Les moyennes par classe sont obtenues avec `rowsum()` et les matrices avec `crossprod()`. Le calcul ne construit donc pas une liste de produits individu par individu. Les résolutions matricielles utilisent des factorisations de Cholesky plutôt qu'une inversion générale.

Les colonnes constantes et les colonnes linéairement dépendantes sont détectées avant la factorisation. Elles sont retirées avec un avertissement explicite. Les variables sont ensuite standardisées sur l'échelle intra-classe, ce qui améliore le conditionnement sans modifier les prédictions de la LDA.

4.2 Variables catégorielles

Une variable factorielle ou caractère est codée par $q-1$ indicatrices, la première modalité étant la référence. La spécification (noms et niveaux) est conservée dans le modèle. À la prédiction, les nouvelles données sont donc encodées dans le même ordre. Un niveau inconnu ne provoque pas de plantage : la ligne concernée est signalée et reçoit une sortie `NA`.

4.3 Sélection de variables

La sélection travaille sur les matrices E et T déjà calculées. Pour une variable candidate, le complément de Schur fournit la contribution conditionnelle. Le test repose sur l'approximation F de Rao associée à la lambda de Wilks. Les variables catégorielles sont toujours traitées comme un bloc de colonnes indicatrices.

1. Initialiser l'ensemble sélectionné selon la stratégie choisie.
2. Calculer la lambda et la p-valeur de chaque candidate.
3. Ajouter la meilleure candidate si elle respecte `alpha_enter`.
4. Pour la stratégie stepwise, retirer la variable la moins utile si sa p-valeur dépasse `alpha_stay`.
5. Arrêter lorsqu'aucune modification n'est possible.

4.4 Validation

`$cv()` réalise une validation croisée stratifiée. Le nombre de plis est automatiquement limité à l'effectif de la plus petite classe afin que chaque apprentissage conserve toutes les classes. La sélection de variables est recalculée dans chaque pli, ce qui évite une fuite d'information.

5 Sorties interprétables

Le modèle expose les coefficients et constantes des fonctions de classification, les probabilités a priori, les effectifs et moyennes de classe, ainsi que les variables et colonnes retenues. La lambda de Wilks $\det(E)/\det(T)$, le test F de Rao, les lambdas partielles, les p-valeurs et les tolérances permettent d'étudier la contribution des variables.

Les axes canoniques sont calculés par décomposition singulière et normalisés pour avoir une variance intra-classe unité. Le modèle expose les valeurs propres, les corrélations canoniques, les coordonnées des individus et les centroïdes de classe. Les performances comprennent la matrice de confusion, l'accuracy, le rappel, la précision, le score F1 et la validation croisée.

6 Application Shiny

L'application permet de charger un fichier tabulé, CSV ou Excel, ou un jeu de données exemple. L'utilisateur choisit la cible, les variables actives et les variables illustratives, puis règle les a priori, la sélection, la taille du jeu test et le nombre de plis. Les résultats sont présentés sous forme de tableaux, graphiques et fonctions de classification.

Le déploiement est disponible pour un individu saisi dans un formulaire et pour un fichier d'individus supplémentaires. Les probabilités par classe peuvent être téléchargées avec la classe prédite. Les erreurs de lecture, les niveaux inconnus et les valeurs manquantes sont affichés sous forme de notifications plutôt que de faire disparaître silencieusement le résultat.

7 Vérification et reproductibilité

Les tests de `testthat` comparent les probabilités et les axes à `MASS::lda()`, la lambda à `stats::manova()`, et couvrent les variables catégorielles, les valeurs manquantes, les colonnes redondantes, la sélection, la validation croisée et les prédictions inconnues.

Les commandes de vérification sont :

```
devtools::document()
devtools::test()
devtools::check()
devtools::build()
```

Le workflow GitHub Actions exécute automatiquement `R CMD check` sur Linux, Windows et macOS. Le tutoriel anglais dans `tutorial/` est reproductible avec les jeux de données fournis dans `inst/extdata/`.

8 Limites et pistes d'amélioration

La méthode suppose une covariance commune aux classes et reste sensible aux échantillons très petits. Pour des données de très grande dimension, une régularisation de la covariance ou une réduction de dimension préalable serait pertinente. Une extension future pourrait proposer un paramètre de shrinkage, des courbes ROC et une calibration des probabilités, tout en conservant l'interface actuelle.

9 Références

— Fisher, R. A. (1936). The use of multiple measurements in taxonomic problems. *Annals of Eugenics*, 7, 179–188.

— Rao, C. R. (1951). An asymptotic expansion of the distribution of Wilks' criterion. *Bulletin of the International Statistical Institute*, 33, 177–180.

— Venables, W. N. and Ripley, B. D. (2002). *Modern Applied Statistics with S*. Springer.

— R Core Team (2026). *R : A Language and Environment for Statistical Computing*. R Foundation for Statistical Computing.